

HelixLLM

HelixLLM

Tagline: One binary, six modes — OpenAI- and Anthropic-compatible inference from your laptop to a multi-host cluster.

Summary HelixLLM is an enterprise-grade distributed LLM system in Go: a single binary with a mode system that scales from single-host development to multi-host production. It serves fully OpenAI- and Anthropic-compatible APIs over HTTP/3, with local llama.cpp inference, a scored multi-provider fallback chain, a RAG pipeline, and a ReAct agent system.

Short description HelixLLM is a single-binary, Go-based distributed LLM system. It exposes OpenAI- and Anthropic-compatible APIs over HTTP/3, runs local llama.cpp inference, auto-discovers and scores free cloud providers into a failover chain, and adds a RAG knowledge pipeline plus a tool-calling ReAct agent — deployable in six modes.

Long description HelixLLM is an enterprise-grade distributed LLM system built in Go with Gin. It compiles to a single binary whose mode system enables flexible deployment: run it as `full` (all-in-one), or split responsibilities across gateway, brain, knowledge, agents, and control modes on multiple hosts.

It provides fully OpenAI- and Anthropic-compatible APIs, so existing SDK clients work unmodified, served over HTTP/3 (QUIC) with automatic HTTP/2 fallback and TLS 1.3. Local inference runs via llama.cpp with CUDA, Metal, and ROCm support. A standout feature is the multi-provider fallback chain: HelixLLM auto-discovers free models from 7+ cloud providers (Chutes, OpenRouter, HuggingFace, Nvidia, Cerebras, SambaNova, Together), scores them via LLMsVerifier (refreshed every 5 minutes), and routes through the ranked chain with automatic 429/5xx failover — with llama.cpp as a guaranteed last resort.

Beyond raw inference, HelixLLM includes a RAG knowledge pipeline (ingestion, chunking, embedding, vector search) and a ReAct agent system with tool calling, conversation sessions, and RAG integration. In `full` mode all layers communicate via direct Go calls with zero network overhead; distributed, the same binary

coordinates over gRPC, SSE, and Kafka. It ships Brotli/gzip negotiation, SSE streaming matching OpenAI/Anthropic formats, API-key and JWT auth with rate limiting, Prometheus metrics, OpenTelemetry tracing, and 43 Go submodules of production infrastructure.

Why we built it Teams need inference that is portable, standards-compatible, and resilient — without rewriting clients or being captive to one provider or one machine. HelixLLM was built so the same binary can run locally for development and scale to a multi-host production cluster, speaking the OpenAI and Anthropic dialects clients already use.

Why it's a game-changer It collapses a stack — gateway, local inference, cloud fallback, RAG, and agents — into one binary with a mode switch, and makes cloud provider reliability a first-class, measured concern via a scored, self-healing fallback chain that always degrades to guaranteed local inference.

What's innovative - A single binary with a six-mode system that runs all-in-one or as distributed roles, using direct Go calls in full mode and gRPC/SSE/Kafka when split. - A scored, auto-discovering multi-provider fallback chain (7+ free providers) ranked by LLMsVerifier with 429/5xx failover and guaranteed llama.cpp last resort. - Dual OpenAI- and Anthropic-compatible surfaces served over HTTP/3 with automatic HTTP/2 fallback. - Local inference across CUDA, Metal, and ROCm from the same codebase.

Biggest technical challenges + how solved - *Scaling from one host to many without a rewrite* — solved with a mode system on one binary: direct in-process calls in full mode, gRPC/SSE/Kafka across distributed modes. - *Unreliable/rate-limited free cloud providers* — solved with auto-discovery, LLMsVerifier scoring, proactive rate-limit header tracking, and automatic failover down to local llama.cpp. - *Client compatibility* — solved by implementing both OpenAI and Anthropic API shapes (including SSE streaming formats) so SDKs work unmodified.

Tech stack - Go + Gin: Chosen for concurrency and a single-binary distribution model; used for the whole system and the gateway HTTP layer. - **HTTP/3 (QUIC) + TLS 1.3, with HTTP/2 fallback:** Chosen for modern low-latency transport; used as the server surface with automatic negotiation. - **llama.cpp (CUDA/Metal/ROCM):** Chosen for portable local inference across GPU/accelerator backends; the guaranteed fallback provider. - **LLMsVerifier:** Chosen to quantify provider quality; used to score and rank the cloud fallback chain (5-minute refresh). - **Cloud providers (Chutes, OpenRouter, HuggingFace, Nvidia, Cerebras, SambaNova, Together):** Chosen to exploit free-tier capacity; auto-discovered and ranked into the failover chain. - **gRPC + SSE + Kafka:** Chosen for inter-mode communication in

distributed deployments. - **Vector store / embeddings:** Chosen to power the RAG knowledge pipeline (ingest, chunk, embed, search). - **Prometheus + OpenTelemetry:** Chosen for metrics and tracing across modes. - **43 Go submodules (vasic-digital ecosystem):** Chosen for reuse of production infrastructure primitives.

Public links - Repo: <https://github.com/HelixDevelopment/HelixLLM> (public; note: the canonical repo currently resolves to <https://github.com/HelixDevelopment/llm> — the HelixLLM path redirects to it). Apache/MIT license not declared in repo metadata (licenseInfo null) — UNVERIFIED.

Suggested diagrams/illustrations 1. Mode-system diagram: one binary shown deploying as full on a laptop vs. gateway/brain/knowledge/agents/control split across cluster hosts. 2. Fallback-chain flow: request → ranked cloud providers (with 429/5xx skip arrows) → guaranteed llama.cpp local fallback, annotated with LLMsVerifier scoring. 3. Compatibility layer: OpenAI and Anthropic client SDKs both hitting the same HTTP/3 gateway. 4. RAG + ReAct agent loop: document ingestion → vector search → agent tool-calling with conversation sessions.

Site relevance Both — company framing on vasic.digital (distributed inference platform) and personal authorship framing on milosvasic.ru.

Priority tier: Helix-primary

Source provenance - gh api repos/HelixDevelopment/llm/readme (primary; fetched live 2026-08-05 — features, six modes, fallback chain, endpoints, architecture, project structure, anti-bluff test posture). - gh repo view HelixDevelopment/HelixLLM / HelixDevelopment/llm --json ... (metadata: public, Go, repo name llm, description “Helix LLM - Local running super model”, pushedAt 2026-07-25, licenseInfo null). - /Volumes/T7/Projects/vasic/_analysis/github-helix-others.md/.json (inventory: HelixLLM, HelixDevelopment org, Go, public, “Local running super model”). - Not available locally (no helix_llm dir under /Volumes/T7/Projects/). - UNVERIFIED: the README’s “85% coverage threshold” and “43 Go submodules” count are self-reported. License is undeclared in metadata.